Personal Finance Tracker

A Browser-Based Financial Management Application

Project Documentation — Pradeep Narsupalli | March 2026

Table of Contents

1. Project Overview
2. My Background — Why This Project Makes Sense for Me
3. Why I Built This
4. Technology Stack & Infrastructure
5. System Architecture
6. Feature Requirements — What I Wanted to Build
7. Development Timeline — How It All Came Together
8. Feature Deep-Dive — All Five Tabs Explained
9. Technical Challenges & How I Solved Them
10. Data Architecture — How the App Stores Everything
11. Design System
12. What I Would Do Differently
13. Future Roadmap
14. Reflections

---

1. Project Overview

The **Personal Finance Tracker** is a fully functional, browser-based financial management application I built to track my income, expenses, investments, and overall net worth — all from one place, without giving any of my data to a third-party service.

The app runs entirely in the browser. There is no backend, no database server, no login screen, and no cloud storage. Everything lives in the browser's local storage, which means my financial data never leaves my device. I built it specifically for the Australian context — AUD currency, ASX stocks, local merchant names in bank statements, and the way Australian banks format and export their CSV files.

This project sits at the intersection of everything I've done in my career — frontend development skills that meant I could specify things precisely, and product thinking that meant I knew what actually needed to be built and why.

Live Repository: https://github.com/pradeepn556/personal-finance-tracker

---

2. My Background — Why This Project Makes Sense for Me

I started my career as a **frontend web developer**. I built UIs, worked with APIs, understood how browsers work, how state management works, why certain technical decisions create problems downstream. That foundation is real and it still informs how I think today.

Over time I moved to the product side — I'm now a **Product Specialist** at a product-based company, which in practice means I do what a Product Manager or Business Analyst would do at other organisations: I define requirements, I own workflows, I validate features, I work between the business and the engineering teams to make sure what gets built is what's actually needed.

The reason I mention this upfront is because it explains why this project looks the way it does. I didn't approach it as a pure product person who had to learn enough to get by. I knew what React was, I knew what CORS was, I knew why localStorage was the right storage choice, I knew why the data model for investment tranches would matter for P&L accuracy. I specified things precisely because I'd built things precisely before.

At the same time, I deliberately chose not to write the code myself. That was intentional — not because I couldn't, but because I wanted to demonstrate a different way of working: **detailed requirements + AI-directed execution + rigorous validation**. This is the model I believe will define how product teams work over the next few years, and this project is my proof of concept.

---

3. Why I Built This

I needed this app. Not as a portfolio exercise — I genuinely needed it. The alternatives I looked at all had one or more of the following problems:

- **Too generic** — not built for Australia. They don't understand ASX stocks, AUD, the way Woolworths shows up in a bank statement, or the difference between how CommBank and ANZ Amex format their CSV exports.
- **Too expensive** — personal finance apps in Australia typically charge $10–$20/month for features I'd only use partially.
- **Privacy concerns** — most tools require linking your bank accounts through open banking APIs, which I wasn't comfortable with. My financial data doesn't need to be on someone else's server.
- **Too shallow** — they track spending but don't connect it to investments, savings rate, and net worth in one coherent view.

So the requirement was clear: build something purpose-built for my situation, that works offline, and keeps everything on my own machine. And since I was building it anyway, I'd build it properly — full feature set, professional design, real data model, real calculations.

---

4. Technology Stack & Infrastructure

I know this stack. I chose it because it's what I'd reach for if I were still writing the code myself — modern, fast, industry-standard, and genuinely good at this type of application.

Frontend Framework

Technology	Version	Why I Chose It
React	19.2	I know the component model well. Declarative, predictable state, and the ecosystem around it is mature.
Vite	7.3	Significantly faster than Create React App for hot-reload and build times. This is what I'd have set up myself.
Tailwind CSS	4.2	Utility-first means you build a design system through constraints, not through fighting CSS specificity. Faster to iterate on design.
Recharts	3.7	React-native charting — no need to drop into D3 directly. Composable and reasonably flexible.

Lucide React	0.577	Consistent, clean icon set. Good API.

Data Storage

Technology	Why I Chose It
Browser localStorage	No server needed. Data stays on the device. Completely private. For a single-user personal app with this data volume, it's the right call — not a workaround.

External APIs (Free Tier)

API	Purpose	Coverage
CoinGecko	Cryptocurrency live prices	No API key needed. CORS-friendly. 50+ coins.
Finnhub	US stocks & ETFs live prices	Free API key. 60 calls/minute. NASDAQ, NYSE.
Twelve Data	ASX stocks & ETFs live prices	Free API key. 800 calls/day. CORS-enabled. `.AX` suffix support.

Why No Backend?

This was a deliberate architectural choice, not a limitation. I don't need:

- A database (localStorage handles the data volume comfortably)
- User authentication (single-user personal app — there's no "other user")
- An API server (all external calls are made directly from the browser)
- Hosting costs (the app runs from any static host, or locally)

Having built backends before, I know when you need one. This isn't that case.

Development & Tooling

Tool	Purpose
Git + GitHub	Version control, portfolio hosting
npm	Package management
ESLint	Code quality linting
Claude Code (AI)	AI-directed implementation — requirements translated to working code

---

5. System Architecture

```
┌──────────────────────────────────────────────────────────┐
│                     Browser (Client Only)                 │
│                                                           │
│  ┌──────────────────────────────────────────────────┐    │
│  │                      React App                    │    │
│  │                                                   │    │
```

```
│  │  ┌──────────┐  ┌──────────┐  ┌──────────────────┐  │   │
│  │  │App.jsx   │  │Shared    │  │  5 Tab Components │  │   │
│  │  │State Hub │  │Components│  │  Dashboard        │  │   │
│  │  │          │  │          │  │  Income           │  │   │
│  │  │income[]  │  │StatCard  │  │  Investments      │  │   │
│  │  │expenses[]│  │Section   │  │  Expenses         │  │   │
│  │  │investm[] │  │          │  │  Settings         │  │   │
│  │  │budgets{} │  │          │  │                   │  │   │
│  │  │settings{}│  │          │  │                   │  │   │
│  │  └──────────┘  └──────────┘  └──────────────────┘  │   │
│  │                                                     │   │
│  │  ┌──────────────────────────────────────────┐      │   │
│  │  │              Utility Layer                 │      │   │
│  │  │  storage.js   calculations.js  formatters.js  │   │
│  │  │  priceFetcher.js               dateHelpers.js │   │
│  │  └──────────────────────────────────────────┘      │   │
│  └────────────────────────────────────────────────────┘   │
│                                                            │
│  ┌──────────────────┐                                      │
│  │   localStorage    │                                      │
│  │  finance_app_*    │                                      │
│  └──────────────────┘                                      │
└────────────────────────────────────────────────────────────┘
          │               │               │
          ▼               ▼               ▼
     CoinGecko API     Finnhub API    Twelve Data API
     (Crypto prices)  (US Stocks)    (ASX Stocks)
```

State Management

All application state lives in `App.jsx` as React `useState` hooks and flows down to child components via props. I deliberately avoided Redux or any external state library — for an app of this size and scope, the added complexity isn't worth it. The trade-off is some prop-drilling, which I was comfortable accepting. I've seen teams reach for Redux too early and spend more time managing the state library than the actual state.

Data Flow

1. On startup, `App.jsx` reads all data from `localStorage` via `loadAllData()`
2. User interactions call `updateData(key, value)` on `App.jsx`
3. `updateData` writes to `localStorage` and updates React state simultaneously
4. Components re-render with the new data

---

6. Feature Requirements — What I Wanted to Build

Before writing a single line of code, I documented a 35-page requirements specification covering every feature in detail. Coming from a frontend background, I wrote this spec differently than a pure product person would — I included data model decisions, API integration approaches, and edge cases that you only think of if you've actually had to build these things before.

Core Principles

- **Privacy first** — no data leaves the device, ever

- **Australian context** — AUD currency, ASX market, local merchants, DD/MM/YYYY dates
- **One-screen visibility** — all key financial metrics visible without hunting
- **No friction** — adding a transaction should take under 30 seconds

Dashboard Requirements

- Net worth prominently at the top — the single most important number
- Monthly income vs expenses vs savings at a glance
- Investment portfolio value with unrealized P&L
- Recent transactions feed combining all categories
- 12-month trend chart for income, expenses, and savings
- Financial health indicators (savings rate, expense ratio, investment allocation %)

Income Requirements

- Track multiple income streams: salary, side hustle, partner income, rental
- Tag income by source and recipient (Me / Partner / Shared)
- Monthly summary with year-to-date totals
- Charts showing income breakdown by source and month-over-month trend
- Support for different pay cycles (weekly, fortnightly, monthly)

Investment Requirements

- Track stocks (ASX + US), ETFs, crypto, bonds, and other assets
- Live price fetching — auto-refresh current prices from market APIs
- Support for multiple purchase lots ("tranches") of the same asset
- Full P&L calculation: unrealized (open positions) and realized (closed positions)
- Holdings heatmap sized by portfolio weight, coloured by P&L
- Partial position close — sell X of N units without closing the entire holding
- Asset allocation pie chart breaking down by asset type
- Support for ANZ.AX, WTC.AX, CBA.AX and other local holdings alongside US stocks

Expense Requirements

- Track expenses against monthly budgets by category
- Preset Australian merchant categories
- Progress bars showing budget consumed per category
- Day/week/month/year filter views
- Bank statement import — upload a CSV and auto-categorise transactions
- Credit card tagging — mark which card each statement belongs to
- Duplicate detection when importing

Settings Requirements

- Customisable currency, date format, and theme
- API key management for Finnhub and Twelve Data
- Credit card / bank account names (used in the import dropdown)
- Budget management — set monthly limits per category
- Data export (JSON backup) and import (restore from backup)
- Clear all data option

---

7. Development Timeline — How It All Came Together

The entire application was built in a single focused session across **5–6 March 2026**. Here is a chronological breakdown of every commit and what changed.

Phase 1 — Project Scaffold

5 March 2026, ~4:45pm `feat: scaffold React + Vite project with full structure`

The initial scaffold set up the Vite + React project, Tailwind CSS, folder structure, and ESLint. Nothing visible yet — just the shell.

Phase 2 — Utility Layer + Navigation Shell + Dashboard

5 March 2026, ~5:27pm `feat: add utility layer, navigation shell and Dashboard tab`

I built the utility layer first because everything else depends on it — and I specified it this way intentionally. Having built React apps before, I know that bolting on a clean utility layer after the fact is painful.

`storage.js` — All localStorage read/write operations. Clean abstraction so no component ever touches `localStorage` directly. Includes `generateId()` for unique entry IDs.

`calculations.js` — Pure financial calculation functions: net worth, P&L, savings rate, expense ratio, monthly trends, budget status. All pure functions — input in, number out. No side effects. Easy to test and reason about.

`formatters.js` — AUD currency formatting, DD/MM/YYYY dates, percentage helpers.

`dateHelpers.js` — Date arithmetic: start/end of month, subtract months. Needed for the trend charts and monthly filters.

The **Dashboard** tab was built as the first full feature:

- Net worth hero card
- Income, expenses, and portfolio summary cards
- 12-month trend chart
- Savings rate and expense ratio indicators
- Recent transactions feed

Phase 3 — All Five Tabs (First Version)

5 March 2026, ~8:46pm `feat: complete all 5 tabs — Income, Investments, Expenses, Settings`

All remaining tabs built in one pass. All five tabs functional — data persisted, charts rendered, calculations correct.

Phase 4 — Complete UI/UX Redesign

5 March 2026, ~10:41pm `feat: complete UI/UX redesign — professional LedgerZero-style`

The first version was functional but looked like a dev prototype. The redesign transformed it into a professional dark-theme fintech-style dashboard.

Having designed and built UIs before, I was specific about what I wanted:

- Dark navy backgrounds (`#0F172A` , `#1E2139`), cyan accent (`#06B6D4`)
- Monospace metric values (the way financial dashboards typically display numbers)
- Consistent 44px button heights across the entire app
- Collapsible forms — hidden by default, slide open on demand

- Alternating table rows, hover highlights, icon-only action buttons
- Smooth CSS transitions for all interactive elements

Every single component was restyled in this commit.

Phase 5 — Expenses Bug Fixes

5 March 2026, ~11:12pm `fix(expenses): correct category budget bars, pay-cycle filter, quick filters`

UAT exposed three issues in Expenses:

- Budget progress bars showing wrong percentages
- Pay-cycle date filter scoping incorrectly
- Quick filter buttons with an off-by-one date issue

All three fixed.

Phase 6 — Live Price Fetching

6 March 2026, ~2:09am `feat(investments): live price fetching via Yahoo Finance + CoinGecko`

Added live market price fetching:

- **CoinGecko** for cryptocurrency (no API key, CORS-friendly)
- **Yahoo Finance** as initial stock source

Auto-fetch on symbol entry + "Refresh All" button.

Phase 7 — Investments Polish

6 March 2026, ~2:29am `feat(investments): auto-fetch price on add, fix tranches modal, move heatmap`

- Auto-fetch current price when symbol is entered (on blur)
- Tranches Modal fixes — clicking a symbol shows all lots
- Heatmap repositioned
- Edit/Delete flow improvements in modal

Phase 8 — Yahoo Finance CORS Fix (Finnhub)

6 March 2026, ~12:15pm `fix: replace Yahoo Finance with Finnhub for live stock prices (CORS fix)`

First major technical problem. Yahoo Finance doesn't send CORS headers. I knew immediately what this was — I'd hit CORS issues in frontend work before. Browser silently blocks the response, code sees a network error, price never populates.

Fix: Replaced with **Finnhub** — proper CORS support, 60 calls/minute free, reliable US stock data.

Phase 9 — Critical Bug Fixes

6 March 2026, ~2:13pm `fix: critical crash + data loss + wrong price bugs in Live Prices`

UAT revealed three bugs after Finnhub was wired up:

1. **App crash** — missing null check when API returned no data
2. **Data loss** — "Refresh Prices" was overwriting the investments array incorrectly (a React functional updater issue I recognised from past experience)
3. **Wrong price displayed** — price not being saved back to the correct field

All three fixed.

Phase 10 — ASX + US Dual-Market Support

6 March 2026, ~2:28pm `feat: support both ASX and US (NASDAQ/NYSE) stocks with USD warning`

I have ASX holdings (ANZ, WiseTech, etc.) and needed Australian market data. Finnhub's free tier doesn't fully cover ASX, so the approach was:

- Symbol ends in `.AX` → try ASX → fallback to Yahoo Finance
- US symbol → Finnhub primary
- If a non-ASX price is returned → show "⚠️ Price may be in USD" warning

---

Phase 11 — Alpha Vantage for ASX (Attempted)

6 March 2026, ~3:02pm `feat: add Alpha Vantage for ASX stock pricing`

Added Alpha Vantage as a dedicated ASX data source. **This didn't work.** Alpha Vantage also has no CORS headers. The API key saved fine (that's just a localStorage write), but the actual price fetches were being blocked by the browser. Superseded by the next commit.

Phase 12 — Twelve Data (The Real ASX Fix)

6 March 2026, ~4:44pm `fix: replace Alpha Vantage with Twelve Data — actual CORS fix for ASX stocks`

Before writing any code this time, I tested CORS headers directly using `curl -I`:

```
Alpha Vantage → No Access-Control-Allow-Origin header ✗
Twelve Data   → Access-Control-Allow-Origin: *        ✓
```

Verified, then built. **Twelve Data**: 800 calls/day free, `.AX` suffix support, clean JSON response. The Settings page was updated and routing in `priceFetcher.js` updated:

- Crypto → CoinGecko
- ASX (`.AX`) → Twelve Data → Finnhub fallback
- US stocks → Finnhub → Yahoo Finance fallback

Phase 13 — Bank Statement CSV Import

6 March 2026, ~4:59pm `feat: add bank statement CSV import with auto-categorization (Expenses tab)`

Design decision: CSV only, not screenshots/OCR. I considered OCR but ruled it out immediately. A misread character on a financial amount — `$1,234` read as `$1.234` — creates silently wrong data that's hard to catch. CSVs are machine-generated and 100% accurate. Every Australian bank supports CSV export.

The import pipeline:

1. User drops a CSV file
2. Parser auto-detects column positions (Date, Amount, Description) by header name matching
3. Sign-convention auto-detection (see Technical Challenges section)
4. Each description matched against 17 Australian merchant rule sets → auto-assigned category
5. Preview table: all detected transactions, categories editable before import
6. Duplicate detection against existing expenses
7. User selects rows, clicks Import

Phase 14 — CSV Sign-Convention Bug + Credit Card Tagging

6 March 2026, ~5:20pm `fix: CSV sign-convention bug + add credit card tagging to import`

The bug: Tested with my actual ANZ Amex statement. Only 2 of 57 transactions imported.

ANZ Amex exports purchases as **positive** amounts. My code had `if (rawAmt > 0) skip` — skipped every purchase.

Fix: 2-pass sign-convention auto-detection. Count positives vs negatives; whichever dominates is the "expense" sign. Works automatically with no user configuration.

Credit card tagging added simultaneously:

- Settings → add card names (e.g., "ANZ Amex", "CommBank Everyday")
- Import modal → select which card the statement is from
- Every imported transaction tagged with `paymentMethod`

Phase 15 — KFC Keyword Fix

6 March 2026, ~6:47pm `Fix KFC keyword matching for underscore-separated bank descriptions`

`'KFC '` (trailing space) doesn't match `'KFC_AU_ROSEHILL'` — ANZ uses underscores as separators. Fix: `'KFC '` → `'KFC'`. Caught via Node.js verification testing against real ANZ data.

---

8. Feature Deep-Dive — All Five Tabs Explained

Tab 1: Dashboard

The Dashboard is read-only — no forms, pure information. The principle here was: open the app, see your financial position immediately, without clicking anything.

Net Worth Card (top, full width) Total income received minus total expenses plus current investment portfolio value. The single most important number. Updated live as data changes.

Secondary Metric Cards

- Monthly spending as a percentage of income
- Portfolio value with unrealized P&L
- Savings rate (%)

Net Worth Trend Chart (full width) 12-month area chart. Pulls from all three data categories.

Cash Flow Chart (50% width) Side-by-side bars for income vs expenses per month. Instantly shows whether you're saving or overspending.

Financial Health Indicators (50% width)

- Savings Rate: (Income - Expenses) / Income × 100
- Expense Ratio: Expenses / Income × 100
- Investment Allocation: Portfolio Value / Net Worth × 100

Recent Transactions (full width) Last 10 transactions across all categories. Date, type badge (colour-coded), description, amount.

Tab 2: Income

Summary Cards (4 across)

- This month's income
- Year-to-date total
- Average monthly income (rolling)
- Highest single income entry with source name

Charts

- Income by Source: pie chart (Salary, Side Hustle, Partner, Rental, etc.)
- Monthly Trend: bar chart, last 12 months

Income Table Date, Source, Amount, Pay Cycle, Notes. Sortable. Edit/Delete.

Add Income Form (collapsible) Fields: Date, Source, Amount, Pay Cycle (weekly/fortnightly/monthly/annual/one-off), Income From (Main Job / Side Hustle / Partner / Other), Recipient (Me / Partner / Shared), Notes.

Tab 3: Investments

The most complex tab. Built around the concept of **investment tranches** — each purchase stored as a separate lot for accurate cost-basis tracking.

Portfolio Summary Cards (4 across)

- Total portfolio value (active holdings at current price)
- Unrealized P&L ($) — current value minus cost basis
- Total cost basis
- Number of active holdings

Holdings Table Symbol, Type, Total Qty, Avg Buy Price, Current Price, Total Cost Basis, Current Value, P&L ($), P&L (%). Colour-coded.

Tranches Modal Clicking any row shows all purchase lots for that symbol:

- Date, Quantity, Purchase Price, Current Value, P&L ($)
- Per-tranche: Edit, Delete, Close Units

Close Units Flow (partial position close) "Close Units" opens inline form: units to sell, sold price, sold date.

- Closing all → marks lot `isClosed=true`
- Closing partial → new closed record for sold portion, original lot quantity reduced

Charts

- Asset Allocation pie (50%) — by asset type
- P&L bar chart (50%) — each holding's P&L %

Holdings Heatmap Visual grid: tile size = portfolio weight, tile colour = P&L% (green/red gradient). Hover for details.

Closed Positions (collapsible) Table of sold lots: Symbol, Qty Sold, Buy Price, Sold Price, Realized P&L ($), Realized P&L (%), Sold Date. [Reopen] option.

Live Price Fetching

- Auto-fetches on symbol entry
- Manual "Refresh All Prices"
- Per-symbol: data source, last updated, stale/fresh badge

Add Investment Form (collapsible) Fields: Date, Symbol (e.g., `BHP.AX`, `AAPL`, `BTC`), Asset Type, Quantity, Purchase Price (auto-fetched), Notes.

Tab 4: Expenses

Budget Overview Card (full width, top) Current month's total spend vs total budget. Large numbers, progress bar, over/under status.

Import CSV Card Entry point for bank statement import. Opens the Import Modal.

4-Chart Grid

- Spending by Category (pie)
- Daily spending trend (line, current month)
- Budget vs Actual by category (grouped bar)
- Top merchants by spend (horizontal bar)

Budget by Category Inline progress bars per category. Spent/budget, percentage bar, status pill (On Track / Warning / Over Budget).

Filter Section Date range, category filter, payment method filter.

Transactions Table Date, Description, Category badge, Amount, Payment Method, Notes. Sortable. Edit/Delete.

Add Expense Form (collapsible)

Import Modal (full-screen) Steps: Drop CSV → Select card/account → Preview & edit categories → Import selected rows.

Tab 5: Settings

General — Currency, date format, theme, charts style.

Live Price API Keys — CoinGecko (no key), Twelve Data (ASX), Finnhub (US). Show/hide toggle, save/clear, status indicator.

Credit Cards & Accounts — Add/edit/delete card names. Used in import dropdown and as `paymentMethod` tags.

Budget Management — Monthly limits per category. Real-time saving.

Data Management — Export (JSON), Import (merge with existing), Clear All.

---

9. Technical Challenges & How I Solved Them

Challenge 1: CORS — The Invisible Wall

What is CORS? Cross-Origin Resource Sharing is a browser security policy that blocks JavaScript from reading responses from external domains unless that domain explicitly allows it via an `Access-Control-Allow-Origin` header.

I'd hit CORS issues in frontend work before, so when stock price fetching stopped working silently — no error visible, just no price — I knew what to look for immediately.

The problem:

- **Yahoo Finance**: No CORS headers → every stock fetch blocked
- **Alpha Vantage**: Also no CORS headers → same problem, just discovered later

The tricky part with Alpha Vantage was that the API key *appeared* to save correctly (because saving a string to localStorage always works), but the actual API calls were being silently blocked by the browser. Without frontend experience, this is very easy to misdiagnose as an API key issue.

How I diagnosed it: Used `curl -I` to inspect HTTP response headers directly — bypassing the browser entirely:

```
curl -I "https://www.alphavantage.co/query?
function=GLOBAL_QUOTE&symbol=BHP.AX&apikey=demo"
# Result: No Access-Control-Allow-Origin header → BLOCKED ✗

curl -I "https://api.twelvedata.com/price?symbol=BHP.AX&apikey=demo"
# Result: Access-Control-Allow-Origin: * → ALLOWED ✓
```

Lesson reinforced: Verify external API CORS headers before building the integration. Don't assume — check. This is something I now always do first.

Final API stack:

Asset Type	Primary	Fallback
Crypto	CoinGecko ✓	Yahoo Finance (CORS-blocked in most browsers)
ASX Stocks (.AX)	Twelve Data ✓	Finnhub → Yahoo Finance
US Stocks	Finnhub ✓	Yahoo Finance

Challenge 2: React Functional Updater — Data Loss on Price Refresh

The bug: After a "Refresh All Prices" action, the investments data would be corrupted or lost.

Root cause: The `updateData` function in `App.jsx` was designed to accept either a value or a functional updater (a function that receives the previous state and returns new state — standard React pattern). The price refresh code was passing a function, but a different part of the save logic was calling `JSON.stringify` on it before storing to localStorage.

`JSON.stringify(function)` returns `undefined`. So the investments key in localStorage was being overwritten with `undefined`, effectively deleting the data.

I recognised this pattern immediately — it's a React state management gotcha I've seen before. The fix was ensuring `updateData` properly detected function vs value arguments before serialising.

Challenge 3: CSV Sign-Convention — Why Only 2 of 57 Transactions Imported

The problem: Uploaded my ANZ Amex statement. Only 2 transactions appeared.

Root cause: ANZ Amex exports purchases as **positive** amounts and payments/refunds as **negative** amounts. Standard bank accounts (CommBank, Westpac, NAB) do the opposite — purchases are negative, credits are positive.

My original code had `if (rawAmt > 0) skip` — which skipped every purchase on the Amex statement.

I ran a Node.js test against the actual CSV to diagnose:

```
Positives found: 57
Negatives found: 2
→ positiveIsExpense = true (ANZ Amex style)
→ Transactions extracted: 57 ✓
```

Fix — 2-pass auto-detection:

```
Pass 1: Count positive vs negative amounts
        Positives dominate → ANZ Amex style (positive = expense)
        Negatives dominate → Standard bank style (negative = expense)

Pass 2: Apply the correct filter
```

No user configuration needed. Works automatically for both formats.

Challenge 4: Keyword Matching — Underscore Separators

The problem: `'KFC '` (trailing space) doesn't match `'KFC_AU_ROSEHILL'`. ANZ uses underscores between merchant name and location.

Why trailing spaces exist: Short keywords like `'BP'` would false-positive match `'AUTOPAY'` or `'BYPASS'`. Trailing space forces a rough word-boundary check.

Fix: `'KFC'` is distinctive enough — no common expense description contains "KFC" without it being the fast food chain. Trailing space removed. Caught via Node.js testing against real ANZ data before it sat in production unnoticed.

Challenge 5: Investment Tranche Data Model

The problem: If I buy 100 shares of CBA at $85, then 50 more at $91, then want to sell 30 — which lot do I sell from? How do I track this without losing cost-basis accuracy?

The solution — per-lot storage: Each purchase is stored as an independent record ("tranche"). Partial close logic:

- Closing ALL units of a tranche → mark `isClosed: true`, record `soldDate` and `soldPrice`

- Closing PARTIAL units → create a **new closed record** for the sold portion, reduce original tranche quantity

This means:

- Cost basis history is never lost
- Every lot's purchase price is preserved separately
- Realized P&L is exact: `(soldPrice − purchasePrice) × qtySold`
- Active portfolio only includes open positions

I specified this model in the PRD because I'd thought through the edge cases before building. A simpler "quantity of symbol" model would have been quicker to build but would break the moment you have multiple buy-ins at different prices.

---

10. Data Architecture — How the App Stores Everything

All data is stored in `localStorage` under five keys:

Key	Type	Description
`finance_app_income`	Array	All income entries
`finance_app_investments`	Array	All investment lots (tranches)
`finance_app_expenses`	Array	All expense transactions
`finance_app_budgets`	Object	Monthly budget limits per category
`finance_app_settings`	Object	User preferences

> ***Note:*** *API keys (Finnhub, Twelve Data) are stored separately in localStorage and deliberately excluded from the JSON export. This prevents accidentally sharing API keys when sending a backup file to someone.*

Income Entry Schema

```
{
  "id": "inc_1741234567890_ab3f",
  "date": "2026-03-01",
  "source": "Salary",
  "amount": 5800.00,
  "payCycle": "fortnightly",
  "incomeFrom": "Main Job",
  "recipient": "Me",
  "notes": "March pay"
}
```

Investment Entry Schema

```
{
  "id": "inv_1741234567890_cd9e",
  "date": "2025-11-15",
```

```
  "symbol": "WTC.AX",
  "type": "ASX Stock",
  "quantity": 25,
  "purchasePrice": 112.50,
  "currentPrice": 139.80,
  "isClosed": false,
  "soldDate": null,
  "soldPrice": null,
  "notes": "Added after earnings"
}
```

Expense Entry Schema

```
{
  "id": "exp_1741234567890_ef7a",
  "date": "2026-03-05",
  "description": "WOOLWORTHS 1106 PARRAMATTA",
  "amount": 124.65,
  "category": "Groceries",
  "paymentMethod": "ANZ Amex",
  "notes": ""
}
```

Settings Schema

```
{
  "currency": "AUD",
  "dateFormat": "DD/MM/YYYY",
  "theme": "dark",
  "chartsStyle": "Professional",
  "cards": ["ANZ Amex", "CommBank Everyday"],
  "notifications": {
    "enabled": true,
    "budgetAlerts": true,
    "overspend": true,
    "investment": true
  }
}
```

Unique ID Generation

```
generateId('inc') → "inc_1741234567890_ab3f"
//                    prefix  timestamp    random 5-char suffix
```

Prefix makes the type immediately obvious. Timestamp + random suffix avoids collisions even within the same millisecond.

---

11. Design System

A consistent visual language applied across every component. I've worked on enough frontend projects to know that an inconsistent design system — different button heights here, different padding there — is death by a thousand cuts. So this was specified upfront.

Colour Palette

Token	Hex	Usage
`bg-primary`	`#0F172A`	Main page background
`bg-card`	`#1E2139`	Card/panel backgrounds
`bg-header`	`#1A2332`	Table headers, section headers
`bg-alt-row`	`#1A2336`	Alternating table rows
`border`	`#334155`	All borders and dividers
`text-primary`	`#F1F5F9`	Main text
`text-muted`	`#CBD5E1`	Secondary text, labels
`text-dim`	`#64748B`	Placeholder, hints
`accent-cyan`	`#06B6D4`	Active states, primary CTAs
`green`	`#10B981`	Positive P&L, on-track budgets
`red`	`#EF4444`	Negative P&L, over-budget
`amber`	`#F59E0B`	Warnings, near-budget

Spacing

- Page padding: 24px
- Section gap: 32px
- Card gap: 16px
- Card internal padding: 20px
- Table cell padding: 12px

Typography

- Metric values: 32px, bold, monospace (financial dashboards use monospace for alignment)
- Card titles: 12px, uppercase, `text-muted`
- Section headings: 18–20px, bold, white
- Table headers: 13px, bold, white

Animations

- Form collapse: `max-height 300ms ease` + `opacity 300ms ease`
- Button hover: `background 150ms ease`
- Input focus: `border + box-shadow 200ms ease` (cyan glow)

Component Conventions

- Forms **collapsible by default** — screen starts clean, forms appear on demand
- Tables **alternating rows** + hover highlight
- Action buttons **icon-only** in tables to save space
- Positive amounts always **green**, negative always **red**, zero muted
- P&L percentages always include `+` sign for positive values

---

12. What I Would Do Differently

1. Test CORS headers before picking any API The Alpha Vantage detour cost time. I knew the CORS concept from frontend work, but I didn't apply that knowledge early enough in the API evaluation step. Going forward: `curl -I` before writing any integration code.

2. Test with real data from day one The sign-convention bug was only caught because I tested with my actual ANZ Amex statement. If I'd only used fake/manually created test data, the bug would have sat hidden until someone tried a real import. Real data surfaces real edge cases that synthetic data never will.

3. Write categorisation rules against real bank descriptions earlier The merchant keyword rules were written from memory (e.g., `'KFC '`). Running them against actual ANZ descriptions during the build phase — not after — would have caught the underscore-separator issue before it made it through.

4. Centralise the design tokens earlier Colour values and spacing lived as inline Tailwind classes across components. Consistent, but not centralised. For a larger project I'd put these in a `theme.js` or CSS custom properties file from the start.

---

13. Future Roadmap

Near Term

- **ASX live prices** — Twelve Data integration is live but UAT is ongoing. Some symbols still returning stale data.
- **Export to CSV** — Currently JSON only. Adding CSV export for accountant/external tool use.
- **Recurring transactions** — Mark expenses as recurring, auto-populate monthly.

Medium Term

- **Partner view** — Data model already supports Me / Partner / Shared tagging. Filtered view to build.
- **Tax year reporting** — Australian financial year is July–June. EOFY summary report.
- **Mobile layout** — Functional on mobile but not optimised. Bottom navigation needed.
- **Open Banking** — Automated import using Australia's Consumer Data Right (CDR), replacing manual CSV.

Long Term

- **PWA** — Installable on iOS/Android as a home screen app, offline support.
- **Multi-currency** — Show US stocks in both USD (purchase price) and AUD (current value via live FX).
- **Notifications** — Browser push for budget alerts, price milestones, monthly summaries.

---

14. Reflections

This project draws on both sides of what I've done professionally.

The frontend background meant I could write a 35-page spec that was specific enough to build from. I didn't write vague requirements like "the investments tab should show portfolio performance." I wrote data model decisions, API routing logic, edge case handling for partial position closes, and CORS architecture choices — because I knew what level of detail you actually need to build something right.

The product side meant I knew what to prioritise. I've worked with development teams long enough to know where complexity hides — and where it doesn't need to exist. The decision to skip Redux, use localStorage over a backend, go CSV-only instead of OCR, store API keys separately from the data export — these are product and architectural decisions, not just technical ones.

The result is an application that I actually use, that handles my real data (ASX holdings, ANZ Amex credit card statements, multi-source income), and that I validated against my own transactions — not a demo dataset.

What I'd say to anyone reviewing this project: the code is one output. The other outputs are the requirements document, the UAT process, the bug escalations with root cause analysis, the architectural decisions and their reasoning. Those are what I'd bring to any product team.

Document prepared by Pradeep Narsupalli — March 2026 Product Specialist | Frontend-turned-Product | AI-Augmented Development Personal Finance Tracker — https://github.com/pradeepn556/personal-finance-tracker